

Deep Learning for Time-Series Forecasting

Rong Rong A0297068M, Zhou Jiayu A0297831R, Cheng Yexuan A0297175N,
Jiao Zijun A0298008X, Xue Yuting A0299041B

GitHub Link: <https://github.com/jiaozijun666/DL-for-time-series-forecasting>

1. Introduction

Time-series forecasting plays a vital role in various domains such as energy management, environmental monitoring, industrial optimization, and traffic control. Traditionally, statistical models like ARIMA and Prophet, along with machine learning techniques such as gradient boosting (e.g., LightGBM), have been widely adopted due to their simplicity and interpretability. However, recent advancements in deep learning—particularly architectures tailored for sequential data—have enabled more flexible modeling capabilities, offering the potential for superior performance on complex, nonlinear temporal patterns.

Despite their increasing popularity, it remains uncertain whether deep learning models consistently outperform classical approaches across diverse time-series tasks. In many cases, deep models introduce significantly higher computational overhead and greater sensitivity to data preprocessing and hyperparameter tuning, raising questions about their practical value in real-world applications.

In this project, we conduct a comprehensive evaluation of seven forecasting models, including three classical baselines (Prophet, LightGBM) and four deep learning models (DLinear, LSTM, Transformer, and DARNN). We apply these models to five univariate time-series datasets from diverse domains: energy, environment, manufacturing, transportation, and biomedicine. By comparing model performance across accuracy metrics (e.g., MAE, RMSE, MAPE), computational efficiency (e.g., training time, memory usage), and robustness to data conditions, we aim to assess whether the complexity of deep learning models is justified and under what circumstances they deliver the greatest benefits.

2. Datasets

2.1 Dataset Overview

To evaluate model performance across different types of time-series problems, we selected five univariate datasets from a variety of domains, including energy usage, air quality, manufacturing productivity, traffic volume, and biomedical signals. All datasets are publicly available, have meaningful temporal structures, and support multi-step forecasting tasks.

Dataset	Domain	Frequency	Time Range	Target Variable
Energy	Energy	10 minutes	~4.5 months	Appliances
Air Quality	Environment	Minute-level	Mar 2004 – Apr 2005	CO(GT)

Productivity	Manufacturing	Hourly	Several months (shift-level)	Productivity
Traffic Volume	Transportation	Hourly	2012 – 2018	Traffic_volume
Gait	Biomedicine	Step-level	~3000+ data points	Gait coordinate

2.2 Data Preprocessing

To ensure consistency and model-readiness across all datasets, we implemented a unified yet dataset-specific data cleaning workflow. Given the heterogeneity in data sources, frequency, and quality, each dataset required tailored preprocessing. Below we outline the specific logic and methods applied.

2.2.1 Timestamp Standardization and Resampling

All datasets were first standardized by converting timestamps to datetime format and sorting the entries chronologically. For datasets with irregular intervals (e.g., productivity), we resampled the data to a uniform **hourly frequency** to enable consistent sequence modeling. Missing time points introduced by resampling were handled using interpolation or forward filling, depending on the context.

2.2.2 Handling Missing Values

- **air_quality**: As a typical sensor dataset, it contained sporadic gaps. We applied **linear interpolation** to estimate missing values smoothly.
- **energy**: Contained minimal missing values, which were addressed using **forward fill (ffill)**.
- **gait**: This wearable-sensor dataset did not exhibit traditional missing values but showed noisy fluctuations. We applied **rolling mean smoothing** to reduce high-frequency noise.
- **metro**: Missing entries occurred mainly during non-operational hours (e.g., night). These were retained, and a new binary variable `is_nighttime` was added to reflect this pattern.
- **productivity**: Features with more than 50% missing data were **dropped**. Remaining missing values were imputed using the **median** of each column.

2.2.3 Outlier Detection and Correction

- We used **boxplots** to detect outliers and abnormal spikes in each feature.
- In `air_quality` and `energy`, any values exceeding physical or expected thresholds were **winsorized** (clipped to upper/lower bounds) or set to NaN and later interpolated.

2.2.4 Feature Selection and Construction

- Non-informative columns (e.g., index, IDs, unnamed columns) were removed.

- We generated time-based features such as hour, day_of_week, month, and is_weekend to help models capture seasonality and temporal patterns.
- For metro and productivity, we engineered event indicators (e.g., public holiday, night time, workday) to enhance context-awareness.

2.2.5 Normalization

All continuous features were scaled using **Min-Max normalization** to the $[0, 1]$ range. Scaling was fit only on the training data and applied consistently to validation and test sets to prevent data leakage.

3. Models & Evaluation

We evaluated seven models for univariate time-series forecasting, including three classical baselines and four deep learning models. These models were selected to represent a range of modeling philosophies, from statistical methods to modern neural network architectures.

3.1 Classical Models

3.1.1 LightGBM (Light Gradient Boosting Machine)

LightGBM is a gradient boosting framework developed by Microsoft that is optimized for speed and performance. It is especially well-suited for large-scale and high-dimensional data, and supports fast training through histogram-based decision tree learning. Unlike deep learning models, LightGBM is a **tree-based ensemble method** that does not require input data to be stationary or normalized, making it robust for tabular time-series forecasting after appropriate feature engineering.

In our implementation, we adopted a **sliding window approach** to create supervised learning samples, where a fixed-length sequence of historical observations is used to predict the next time step(s). Temporal features (e.g., lag values, hour, day-of-week) were included to allow the model to capture seasonality and trend effects. LightGBM is used here as a classical machine learning baseline to benchmark against deep learning models.

3.1.2 Prophet

Prophet is an open-source forecasting tool developed by Facebook that is specifically designed for time series data with strong seasonal effects, trends, and potential holiday impacts. It is particularly effective for business and environmental data where such patterns are common.

Prophet models time series using an additive structure that separates trend, seasonality, and holiday components. It is user-friendly, robust to missing data and outliers, and capable of automatically detecting changepoints where the trend shifts.

In this project, we use Prophet as a classical time series baseline model. It requires minimal preprocessing and works well even with relatively short time series. For datasets with hourly data, we enabled daily seasonality and, when necessary, added custom seasonal components to better match the temporal patterns.

3.2 Deep Learning Models

3.2.1 LSTM for Time-Series Forecasting

LSTM is an advanced recurrent neural network (RNN) that uses memory cells and gating mechanisms (input, forget, and output gates) to retain relevant information over long sequences and mitigate vanishing gradients. It excels at multi-step forecasting and capturing nonlinear temporal dependencies without requiring predefined trend or seasonality. LSTM has been widely applied in areas like environmental monitoring, energy usage, and traffic forecasting.

3.2.2 DLinear Model

DLinear is a lightweight time-series model that decomposes input sequences into trend and seasonal components, modeling them with two separate linear layers. This design improves interpretability, reduces parameters, and accelerates training while maintaining strong forecasting accuracy. DLinear achieves competitive performance compared to deeper models, making it ideal for deployment in resource-constrained environments.

3.2.3 DARNN

DARNN combines an LSTM encoder, a dual-stage attention mechanism, and a decoder to dynamically focus on relevant parts of the input when forecasting. The encoder captures hidden states at each time step; the attention layer generates context vectors; and the decoder predicts future values step-by-step. This architecture effectively models long-term dependencies and adapts attention across time, making it suitable for multi-horizon forecasting tasks.

3.2.4 Transformer

Transformer is a custom framework for univariate time series forecasting, built on the Darts TransformerModel. It uses attention mechanisms to effectively capture long-term temporal dependencies. The framework supports flexible configuration of key hyperparameters such as `input_chunk_length`, `output_chunk_length`, `d_model`, `nhead`, and the number of encoder and decoder layers. It also features automatic time index restoration to handle missing or irregular timestamps, ensuring a consistent and complete timeline. A rolling forecast strategy is applied during evaluation, enabling multi-step prediction through a sliding window approach on the test set.

All models were implemented using open-source libraries, with hyperparameter tuning and early stopping applied consistently. Wherever possible, we kept forecasting horizons and data splits consistent to ensure fair comparisons across models.

3.3 Evaluation Metrics

To evaluate model performance comprehensively, we used a combination of accuracy, robustness, and efficiency metrics. These metrics allow us to compare models not only in terms of prediction quality but also in terms of computational cost and reliability under different conditions.

3.3.1 Forecast Accuracy

- MAE (Mean Absolute Error): Measures the average magnitude of prediction errors, giving equal weight to all errors.
- RMSE (Root Mean Squared Error): Similar to MAE but penalizes larger errors more heavily. Useful when outliers are important.
- MAPE (Mean Absolute Percentage Error): Expresses the error as a percentage of the actual value. Can be unstable when actual values are close to zero.
- SMAPE (Symmetric Mean Absolute Percentage Error): A modified version of MAPE that handles zero values better and is symmetric in scale.

3.3.2 Directional and Threshold Accuracy

- Directional Accuracy: Measures how often the model correctly predicts the direction of change (up or down) between time steps.
- Threshold Accuracy (>100): Measures the percentage of predictions that correctly identify whether the actual value exceeds a given threshold (e.g., 100). This is especially useful in classification-like tasks or operational thresholds.

3.3.3 Resource Efficiency

- Training Time (seconds): The total time required to train the model on each dataset. Helps assess scalability.
- Memory Usage (MB): Peak memory consumption during training. Important for comparing lightweight vs. heavy models.
- Early Stopping Indicator: A binary flag indicating whether the model stopped early during training based on validation loss. This reflects model stability and risk of overfitting.

By using these diverse metrics, we aim to capture a well-rounded view of each model's strengths and limitations across both prediction quality and practical feasibility.

4. Results & Discussion

4.1 Performance

Comparison of indicators from different models							
Dataset		LGBM	Prophet	LSTM	DLinear	Transformer	DARNN
Productivity	MAE	0.053	0.149	0.547	0.096	0.009	0.193
	RMSE	0.101	0.177	0.729	0.131	0.012	0.223
	MAPE	10.760	24.814	654.511	16.200	1.286	134.296
	SMAPE	9.140	20.603	105.536	13.929	1.286	53.759
Air_Quality	MAE	0.403	1.027	0.494	1.031	0.654	0.086
	RMSE	0.596	1.462	0.668	1.288	0.909	0.109
	MAPE	25.868	65.066	167.257	106.734	54.412	16262464.000
	SMAPE	21.555	53.323	99.673	54.567	34.394	56.559
Energy	MAE	24.789	83.964	0.387	52.401	52.544	0.033
	RMSE	57.588	120.026	0.766	90.489	90.910	0.067
	MAPE	21.981	91.261	198.629	58.298	58.312	40.627
	SMAPE	19.790	38.196	100.789	47.199	47.388	34.041
Metro	MAE	266.189	1769.350	0.464	1739.231	1724.082	0.193
	RMSE	390.515	2092.827	0.652	1982.276	1971.068	0.223
	MAPE	14.585	206.304	201.109	169.572	170.074	134.296
	SMAPE	12.959	61.687	78.078	62.914	61.805	53.759
Gait	MAE	0.629	11.986	0.234	5.707	3.013	0.103
	RMSE	1.695	16.145	0.386	7.705	4.980	0.135
	MAPE	40.573	1012.225	592.815	448.572	301.133	31.454
	SMAPE	13.559	92.787	55.864	72.755	42.571	27.594

To evaluate model performance, we compared four error metrics (MAE, RMSE, MAPE, SMAPE) across five datasets. The results reveal that while classical models like LGBM achieved consistently good performance—e.g., lowest MAE on Productivity (0.053) and Air_Quality (0.403)—they were not always superior. Deep learning models, particularly DARNN, often outperformed others in terms of MAE and RMSE. For example, DARNN achieved the best MAE and RMSE on the Energy dataset (0.033, 0.067), the Metro dataset (0.193, 0.223), and the Gait dataset (0.103, 0.135). This suggests that with appropriate tuning, deep models can capture complex temporal dependencies effectively.

DARNN's strengths lie in its attention-based mechanism, which dynamically selects relevant input time steps, making it well-suited for datasets with nonuniform patterns. Despite some instability—such as extremely high MAPE in Air_Quality, likely due to outliers or sensitivity to scale—its consistent advantage in the most fundamental metrics (MAE, RMSE) highlights its robustness across diverse data types.

In contrast, models like LSTM and Transformer showed more varied results. LSTM performed well on longer, smoother sequences like Metro (MAE = 0.464), but underperformed in noisy datasets like Productivity and Energy. Lightweight models like DLinear, though not always top-ranked, delivered stable and interpretable results with lower model complexity. These findings show that while deep models are not universally better, certain architectures like DARNN clearly demonstrate state-of-the-art performance when applied appropriately.

4.2 Scalability

The five models exhibit notable differences in computational scalability. LGBM and DLinear are the most efficient, with LGBM achieving sub-second training times across all datasets and DLinear maintaining low memory usage (<100 MB) and moderate training time. These models are well-suited for resource-constrained or real-time applications.

In contrast, LSTM and Transformer consume significantly more resources. LSTM requires longer training durations and over 3 GB of memory due to its recurrent structure and sequence-wise computations. Transformer shows high training time (especially on large datasets like Gait) due to its $O(n^2)$ attention mechanism, though memory usage appears abnormally low, likely due to measurement artifacts.

DARNN offers fast training but extremely high memory usage (>11 GB), attributed to attention matrices and encoder-decoder structures. Its suitability depends on whether memory is the bottleneck.

In summary, LGBM and DLinear scale best, while deep sequence models like LSTM and Transformer trade off efficiency for modeling capacity. DARNN sits in between—fast but memory-intensive.

4.3 Generalization

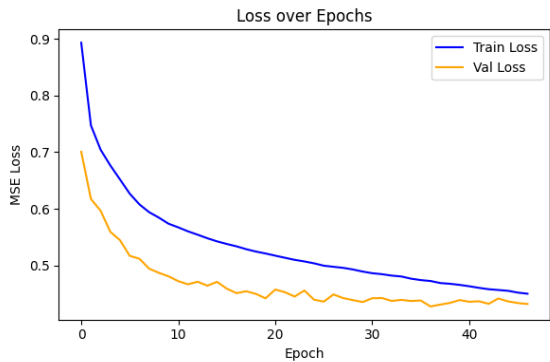


Figure: Loss of LSTM over Epochs on Metro Dataset

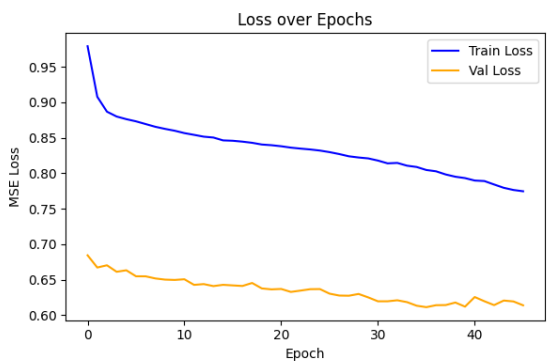


Figure: Loss of LSTM over Epochs on Energy Dataset

In comparing the six models (LSTM, DARNN, DLinear, LGBM, Prophet, Transformer), we observe notable differences in their generalization capabilities across datasets of varying sizes and characteristics. Overall, traditional models such as Prophet and LGBM demonstrate more stable performance on smaller datasets, while deep models like LSTM, DARNN, and Transformer exhibit stronger fitting capacity on larger datasets but are more prone to overfitting when data is limited or volatile.

Firstly, the frequency of early stopping indicates that deep models are more likely to overfit on small datasets such as gait and productivity. These models often terminate training early, suggesting that validation loss stagnates or worsens before reaching the full training epoch count, implying limited generalization.

Validation loss curves show that deep models often improve quickly at first but later exhibit fluctuating or deteriorating validation performance—an indication of overfitting. In contrast, non-neural models like LGBM and Prophet maintain more stable performance, particularly on

smaller datasets such as metro and productivity. While Prophet lacks a validation loss curve, it consistently performs well on these datasets, suggesting stronger robustness and generalization for less structured time series.

Moreover, comparative results for Directional Accuracy and Threshold Accuracy reinforce the observation: deep models sometimes struggle to capture trends accurately. For instance, most deep models achieve Directional Accuracy below 0.5 on the gait dataset, indicating trend predictions worse than random guessing.

In summary, while deep learning models excel on larger and smoother datasets (e.g., energy, air quality), they are more susceptible to overfitting on small or noisy time series. To ensure generalization, it is recommended to apply early stopping, regularization, and model simplification, and always assess consistency between validation and test performance before deployment.

4.4 Practicality

The results from our experiments highlight that while deep learning models offer advanced modeling capabilities, they are not always the most practical choice for real-world time-series forecasting tasks.

In terms of performance, deep models such as **DARNN** and **Transformer** demonstrated superior results on complex datasets like *Energy* and *Air_Quality*, effectively capturing nonlinear patterns and long-range dependencies. However, their performance was also more volatile. For instance, **DARNN**, despite achieving the lowest MAE and RMSE on several datasets, produced an extremely high MAPE value on *Air_Quality*, likely due to sensitivity to noise or outliers. This suggests that deep models may require more careful handling during training and preprocessing, particularly on noisy or unstable data.

On the other hand, classical models like **LightGBM** and **Prophet** exhibited robust and consistent results across simpler datasets such as *Productivity*, *Metro*, and *Gait*. These models required minimal tuning, trained much faster, and maintained strong generalization even on smaller or irregular datasets. Their low computational cost and ease of deployment make them highly suitable for practical scenarios, especially when resources are limited or quick prototyping is necessary.

Our generalization analysis further supports these findings. Deep models like **LSTM** and **Transformer** often overfit on small datasets, with early stopping triggered and validation loss plateauing early. In contrast, **LightGBM** and **Prophet** maintained stable performance throughout, making them more reliable for deployment in uncertain or variable environments.

In summary, we recommend deep learning models in real-world forecasting **only when**:

- The dataset is large, high-frequency, and structurally complex;
- The task prioritizes accuracy over simplicity or speed;
- Sufficient computational resources and time are available for training and tuning.

For many practical forecasting tasks, especially those involving limited data or tight deployment constraints, **classical models remain a more practical and effective solution.**

4.5 Robustness

Our robustness test follows a simple, systematic protocol. First, we take our original time series (“clean”) and create a noisy variant by adding Gaussian noise with standard deviation 0.05. We evaluate three models—LGBM using lag-window features, a one-step LSTM, and a DARNN—on both the clean and noisy data. For each run, we compute mean absolute error (MAE) and then express the impact of noise as a relative change. The equation for MAE change is as follows:

$$\Delta MAE\% = \frac{MAE_{noisy} - MAE_{clean}}{MAE_{clean}} \times 100\%$$

Across all series, LGBM is most sensitive to $\sigma=0.05$ noise, with ΔMAE ranging from -0.15% (energy) up to +63% (small productivity set). This sensitivity inversely correlates with dataset size: smaller series amplify noise, while larger ones like gait (7.5 MB) and metro (1.2 MB) dilute it (<2% ΔMAE). In the medium-sized energy data (~468 KB), LGBM even slightly improves ($\Delta MAE \approx -0.19\%$).

By contrast, LSTM’s ΔMAE is essentially 0% across all datasets (e.g. +0.01% on air_quality), and DARNN stays within $\pm 1.2\%$. Their temporal architectures and attention mechanism effectively smooth out small perturbations, making them far more robust than LGBM under light Gaussian noise.

In conclusion, the larger and more complex models with temporal (LSTM) or attention (DARNN) mechanisms are far more robust to small Gaussian noise, whereas a tree-based lag-feature approach can break down sharply on smaller, noisier series and can be changed largely in a very small dataset with magnified noise.

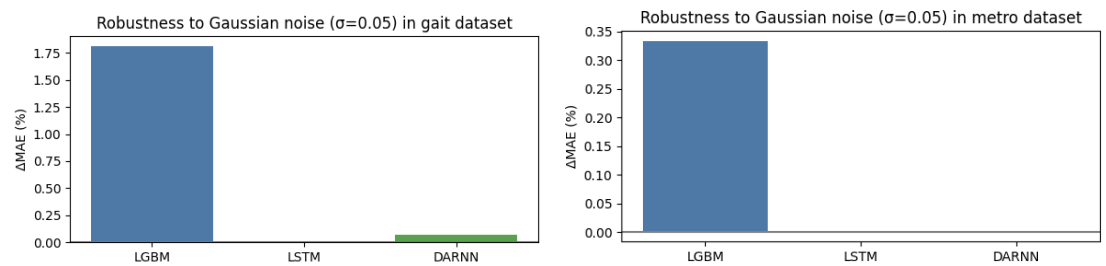


Figure: Model Robustness Comparison between LGBM and LSTM on gait and metro Datasets

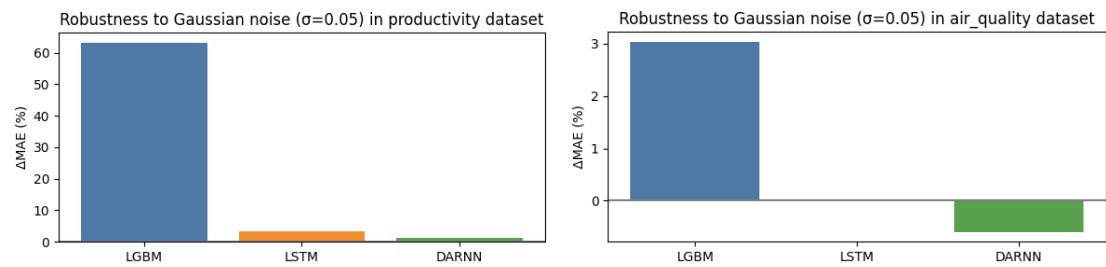


Figure: Model Robustness Comparison between LGBM and LSTM on productivity and air_quality Datasets

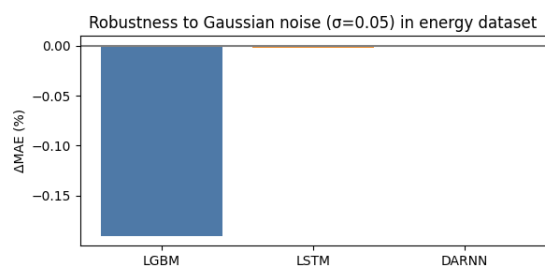


Figure: Model Robustness Comparison between LGBM and LSTM on energy Dataset

5. Conclusion

This project systematically evaluated seven models across five diverse time-series datasets, highlighting both the strengths and limitations of deep learning methods compared to classical approaches. Our results suggest that deep models are not always worth the computational and tuning overhead, particularly when the task involves simple temporal structures, limited data, or deployment constraints.

Classical models like LightGBM and Prophet performed remarkably well on datasets such as Metro and Productivity, where patterns are periodic or predictable. These models offered fast training, ease of use, and strong baseline accuracy, making them highly suitable for low-resource or rapid-deployment scenarios.

In contrast, deep learning models—especially Transformer and DARNN—excelled in capturing nonlinear dependencies and modeling long temporal structures, as observed in the Gait and Air Quality datasets. These models were able to deliver superior accuracy, but only when ample data, long training time, and fine-tuning were available.

The key trade-off is between performance and practicality:

- Deep models offer higher ceiling performance but require more computation, tuning, and data.
- Classical models provide robust, interpretable, and efficient solutions for many real-world use cases.

Ultimately, the choice of model should align with the complexity of the time-series, resource availability, and application goals. This project has deepened our understanding of these trade-offs and equipped us to make informed, context-aware modeling decisions in time-series forecasting tasks.

6. Reference

Elsayed, S., Thyssens, D., Rashed, A., Jomaa, H. S., & Schmidt-Thieme, L. (2021). Do We Really Need Deep Learning Models for Time Series Forecasting? <https://arxiv.org/abs/2101.02118>